

PyTorch Object Detection Interview Code Modification Guide

How to adapt a Faster R-CNN example into a Cat/Dog object detection task

Core idea: In an interview, you do not need to memorize a full detection pipeline from scratch. You need to know which parts of the template must be modified flexibly: dataset path, annotation parser, class labels, number of classes, model head, hyperparameters, inference visualization, and checkpoint metadata.

1. What your current example code does

- Your example is a torchvision Faster R-CNN ResNet50-FPN object detection template originally designed for the PennFudan pedestrian dataset.
- The original code reads images and segmentation masks, converts each mask into bounding boxes, trains Faster R-CNN, and visualizes predicted boxes.
- For a cat/dog detection interview task, the model and training loop can mostly stay the same. The most important changes are the Dataset class and the class definitions.

Module	Current example	How to modify for cat/dog detection
DATA_ROOT / dataset_dir	Downloads and reads PennFudanPed	Point to the provided Data folder
PennFudanDataset	Reads PNGImages and PedMasks	Read images and bounding-box annotations from XML / YOLO TXT / COCO JSON
NUM_CLASSES	2 = background + pedestrian	3 = background + cat + dog
labels	All labels are set to 1	Use cat=1 and dog=2; keep 0 for background
build_model()	Replaces Faster R-CNN predictor	Keep it, but pass num_classes=3
train_one_epoch()	Uses loss_dict returned by the detection model	Mostly keep unchanged
collate_fn()	Handles variable number of boxes	Must keep it
run_inference()	Displays pedestrian text	Replace with CLASS_NAMES[label]
save_checkpoint()	Stores pedestrian class name	Store cat/dog class names

2. First interview step: inspect the dataset and annotation format

What to say in the interview: I will first inspect the dataset structure and annotation format, because the custom Dataset class depends on whether the labels are VOC XML, YOLO TXT, COCO JSON, or segmentation masks.

```
import os

print(os.listdir("Data"))
print(os.listdir("Data/images")[:5])
# Check the actual annotation folder name: annotations / labels / masks
print(os.listdir("Data/annotations")[:5])
```

- .xml usually means VOC annotation; parse xmin, ymin, xmax, ymax.
- .txt usually means YOLO annotation; convert normalized x_center, y_center, width, height to xmin, ymin, xmax, ymax.
- .json may mean COCO annotation; read annotations by image_id.

- .png masks mean you can extract boxes from masks, similar to the PennFudan example.
- If the dataset only gives image-level cat/dog labels without bounding boxes, it is classification, not object detection.

3. Global variables you should modify

```
DATA_ROOT = "Data"
OUTPUT_DIR = "outputs_catdog_detection"
CHECKPOINT_PATH = os.path.join(OUTPUT_DIR, "fasterrcnn_catdog.pth")

CLASS_NAMES = ["background", "cat", "dog"]
NUM_CLASSES = len(CLASS_NAMES) # 3: background + cat + dog

BATCH_SIZE = 1 # Recommended for CPU; use 2 or 4 if GPU is available
NUM_EPOCHS = 1 # For interview debugging, use 1-3 epochs
LEARNING_RATE = 0.005
NUM_WORKERS = 0 # More stable on Windows / Jupyter
SCORE_THRESHOLD = 0.5
TRAIN_RATIO = 0.8
```

Important explanation: In Faster R-CNN, label 0 is reserved for background. Therefore, actual object classes start from 1: cat=1 and dog=2. This is why NUM_CLASSES must be 3, not 2.

4. Required target format from the Dataset class

```
target = {
    "boxes": boxes, # FloatTensor[N, 4], format: [xmin, ymin, xmax, ymax]
    "labels": labels, # Int64Tensor[N], e.g. cat=1, dog=2
    "image_id": image_id, # Tensor[1]
    "area": area, # Tensor[N]
    "iscrowd": iscrowd # Int64Tensor[N]
}
```

- boxes must use [xmin, ymin, xmax, ymax], not YOLO-style [x_center, y_center, width, height].
- labels must be torch.int64.
- boxes must be torch.float32.
- Each image can have a different number of objects, so the DataLoader must use a custom collate_fn.

5. If the interview dataset uses VOC XML annotations

```
import xml.etree.ElementTree as ET

class CatDogVOCDataset(torch.utils.data.Dataset):
    def __init__(self, root, transforms=None):
        self.root = root
        self.transforms = transforms
        self.img_dir = os.path.join(root, "images")
        self.ann_dir = os.path.join(root, "annotations")
        self.imgs = sorted([
            f for f in os.listdir(self.img_dir)
            if f.lower().endswith((".jpg", ".jpeg", ".png"))
        ])
        self.label_map = {"cat": 1, "dog": 2}
```

```

def __getitem__(self, idx):
    img_name = self.imgs[idx]
    img_path = os.path.join(self.img_dir, img_name)
    xml_name = os.path.splitext(img_name)[0] + ".xml"
    ann_path = os.path.join(self.ann_dir, xml_name)

    img = Image.open(img_path).convert("RGB")
    tree = ET.parse(ann_path)
    xml_root = tree.getroot()

    boxes, labels = [], []
    for obj in xml_root.findall("object"):
        class_name = obj.find("name").text.lower().strip()
        if class_name not in self.label_map:
            continue
        bbox = obj.find("bndbox")
        xmin = float(bbox.find("xmin").text)
        ymin = float(bbox.find("ymin").text)
        xmax = float(bbox.find("xmax").text)
        ymax = float(bbox.find("ymax").text)
        boxes.append([xmin, ymin, xmax, ymax])
        labels.append(self.label_map[class_name])

    boxes = torch.as_tensor(boxes, dtype=torch.float32)
    labels = torch.as_tensor(labels, dtype=torch.int64)
    image_id = torch.tensor([idx])
    area = (boxes[:, 3] - boxes[:, 1]) * (boxes[:, 2] - boxes[:, 0])
    iscrowd = torch.zeros((len(boxes),), dtype=torch.int64)

    target = {"boxes": boxes, "labels": labels, "image_id": image_id,
              "area": area, "iscrowd": iscrowd}

    if self.transforms is not None:
        img, target = self.transforms(img, target)
    return img, target

def __len__(self):
    return len(self.imgs)

```

6. If the interview dataset uses YOLO TXT annotations

```

class CatDogYOLODataset(torch.utils.data.Dataset):
    def __init__(self, root, transforms=None):
        self.root = root
        self.transforms = transforms
        self.img_dir = os.path.join(root, "images")
        self.label_dir = os.path.join(root, "labels")
        self.imgs = sorted([
            f for f in os.listdir(self.img_dir)
            if f.lower().endswith((".jpg", ".jpeg", ".png"))
        ])

    def __getitem__(self, idx):
        img_name = self.imgs[idx]
        img_path = os.path.join(self.img_dir, img_name)
        label_name = os.path.splitext(img_name)[0] + ".txt"
        label_path = os.path.join(self.label_dir, label_name)

```

```

img = Image.open(img_path).convert("RGB")
width, height = img.size
boxes, labels = [], []

with open(label_path, "r") as f:
    for line in f.readlines():
        values = line.strip().split()
        if len(values) != 5:
            continue
        class_id, x_center, y_center, box_w, box_h = map(float, values)

        x_center *= width
        y_center *= height
        box_w *= width
        box_h *= height

        xmin = x_center - box_w / 2
        ymin = y_center - box_h / 2
        xmax = x_center + box_w / 2
        ymax = y_center + box_h / 2

        boxes.append([xmin, ymin, xmax, ymax])
        labels.append(int(class_id) + 1) # YOLO 0/1 -> Faster R-CNN 1/2

boxes = torch.as_tensor(boxes, dtype=torch.float32)
labels = torch.as_tensor(labels, dtype=torch.int64)
image_id = torch.tensor([idx])
area = (boxes[:, 3] - boxes[:, 1]) * (boxes[:, 2] - boxes[:, 0])
iscrowd = torch.zeros((len(boxes)), dtype=torch.int64)

target = {"boxes": boxes, "labels": labels, "image_id": image_id,
          "area": area, "iscrowd": iscrowd}

if self.transforms is not None:
    img, target = self.transforms(img, target)
return img, target

def __len__(self):
    return len(self.imgs)

```

7. Model part: keep it mostly unchanged, only change NUM_CLASSES

```

from torchvision.models.detection import fasterrcnn_resnet50_fpn
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

def build_model(num_classes):
    model = fasterrcnn_resnet50_fpn(weights="DEFAULT")
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    return model

model = build_model(num_classes=NUM_CLASSES)
model.to(device)

```

Interview explanation: I use a pretrained Faster R-CNN ResNet50-FPN model and replace the ROI box predictor so that the final classifier outputs background, cat, and dog.

8. Training loop: no separate criterion is needed

```
loss_dict = model(images, targets)
losses = sum(loss for loss in loss_dict.values())
```

```
optimizer.zero_grad()
losses.backward()
optimizer.step()
```

- In training mode, torchvision Faster R-CNN receives images and targets and automatically returns a detection loss dictionary.
- Typical losses include `loss_classifier`, `loss_box_reg`, `loss_objectness`, and `loss_rpn_box_reg`.
- Therefore, you do not need to manually define `nn.CrossEntropyLoss()`, unlike a classification task.

9. Keep `collate_fn` in the `DataLoader`

```
def collate_fn(batch):
    return tuple(zip(*batch))

train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS,
    collate_fn=collate_fn
)
```

Why this is needed: In object detection, each image may contain a different number of bounding boxes. The default `DataLoader` collate function cannot stack variable-length targets into a fixed tensor.

10. Inference visualization: replace pedestrian with cat/dog names

```
CLASS_NAMES = ["background", "cat", "dog"]

label_texts = [
    f"{CLASS_NAMES[label.item()]} {score:.2f}"
    for label, score in zip(labels, scores)
]
```

- Your current example hard-codes the word `pedestrian`. For cat/dog detection, use `CLASS_NAMES[label.item()]`.
- A score threshold of 0.5 is a reasonable starting point. If no boxes are displayed during debugging, temporarily reduce it to 0.3.

11. Checkpoint: save correct class metadata

```
torch.save(
    {
        "epoch": epoch,
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "loss_history": loss_history,
        "num_classes": NUM_CLASSES,
```

```

        "class_names": CLASS_NAMES,
    },
    CHECKPOINT_PATH
)

```

12. Lines most often changed in main()

```

dataset_dir = DATA_ROOT

full_dataset = CatDogVOCDataset(
    root=dataset_dir,
    transforms=get_transform(train=True)
)

test_dataset = CatDogVOCDataset(
    root=dataset_dir,
    transforms=get_transform(train=False)
)

model = build_model(num_classes=NUM_CLASSES)

```

If the dataset uses YOLO format: Replace CatDogVOCDataset with CatDogYOLODataset. The training loop, model construction, optimizer, and scheduler can stay mostly unchanged.

13. Interview plan you can say directly

1. First, I will inspect the dataset folder and annotation format to decide how to implement the Dataset class.
2. Then I will define the class mapping. For Faster R-CNN, label 0 is background, so cat and dog will be labels 1 and 2.
3. Next, I will implement a custom Dataset that returns an image and a target dictionary, where target contains boxes, labels, image_id, area, and iscrowd.
4. I will use a pretrained Faster R-CNN ResNet50-FPN model and replace the box predictor for three classes.
5. During training, the torchvision detection model returns a dictionary of losses, so I will sum the losses and backpropagate.
6. Finally, I will run inference on validation images, filter predictions by confidence score, and visualize predicted boxes with class names and scores.

14. Common mistakes to avoid

Mistake	Correct approach
Setting NUM_CLASSES = 2 for cat/dog detection	Use NUM_CLASSES = 3 because background is included
Using cat=0 and dog=1	Use background=0, cat=1, dog=2
Passing YOLO boxes directly to Faster R-CNN	Convert YOLO boxes to [xmin, ymin, xmax, ymax]
Removing collate_fn	Keep collate_fn because each image has a variable number of objects
Returning only image-level class labels	Object detection requires bounding boxes and labels
Making labels float tensors	labels must be int64 tensors
Using a large batch size on CPU	Use batch_size=1 and 1-3 epochs for a quick CPU check

15. Three code blocks you should remember

A. Class definition

```
CLASS_NAMES = ["background", "cat", "dog"]  
NUM_CLASSES = len(CLASS_NAMES)
```

B. Model head replacement

```
model = fasterrcnn_resnet50_fpn(weights="DEFAULT")  
in_features = model.roi_heads.box_predictor.cls_score.in_features  
model.roi_heads.box_predictor = FastRCNNPredictor(in_features, NUM_CLASSES)
```

C. Detection loss calculation

```
loss_dict = model(images, targets)  
losses = sum(loss for loss in loss_dict.values())  
losses.backward()  
optimizer.step()
```